

FROM BUTTON CLICK TO PIXELS

The production request lifecycle with Next.js + FastAPI

A deep engineering reference for understanding events, rendering, HTTP, CORS, proxies, FastAPI, databases, errors, and performance.

Core mental model

At every boundary, ask: Where is this code running? What data crosses the boundary? What should happen next? What evidence proves that it actually happened?

```
USER INPUT
-> Browser / DOM event
-> React handler
-> fetch() / axios
-> Browser security + cookies + CORS
-> DNS -> TCP/QUIC -> TLS -> HTTP
-> CDN / reverse proxy / load balancer
-> Uvicorn -> ASGI -> FastAPI
-> middleware -> auth -> routing -> validation
-> endpoint -> service -> database
-> HTTP response -> browser
-> Promise -> JSON -> state update
-> React render -> DOM -> layout -> paint -> pixels
```

Use this document as a debugging map: find the first boundary where reality differs from your expectation.

Contents

Part I - The full journey

- 1. One click, end to end
- 2. Before the click: page delivery and hydration
- 3. The physical click, hit testing, DOM events
- 4. React event handling and the JavaScript event loop

Part II - The browser-to-server journey

- 5. fetch(), URLs, environment variables
- 6. Origins, CORS, preflight, cookies
- 7. DNS, transport, TLS, HTTP

Part III - Production infrastructure and FastAPI

- 8. Reverse proxies and load balancers
- 9. Uvicorn, ASGI, middleware, routing
- 10. Dependencies, authentication, validation
- 11. Endpoint, service, repository, database

Part IV - The trip back to the screen

- 12. Serialization and the HTTP response
- 13. fetch response behavior and JSON parsing
- 14. React state, rerenders, reconciliation
- 15. Browser layout, paint, compositing

Part V - Production debugging

- 16. Common React request bugs
- 17. Forms and hidden browser behavior
- 18. HTTP status-code map
- 19. A systematic debugging algorithm
- 20. Performance: where the time actually goes
- 21. Next.js as a middle layer
- 22. Error-by-layer cheat sheet

Part I - The full journey

1. One click, end to end

Assume a Next.js frontend and a FastAPI backend. The frontend has a button that requests users from an API:

```
"use client";

async function loadUsers() {
  const response = await fetch("https://api.example.com/users");
  const data = await response.json();
  setUsers(data);
}

<button onClick={loadUsers}>Load users</button>
```

The FastAPI side contains:

```
@app.get("/users")
async def get_users():
    return [{"id": 1, "name": "Saad"}]
```

At a high level, the journey is simple:

```
click -> loadUsers() -> fetch() -> FastAPI -> database -> response -> setUsers() -> rerender
```

In production, however, each arrow hides a subsystem. A realistic pipeline is:

```
Mouse/finger -> OS -> browser renderer -> DOM event -> React event handler
-> fetch() -> browser Fetch API -> security checks -> DNS -> connection -> TLS -> HTTP
-> CDN/proxy/load balancer -> Uvicorn -> ASGI -> middleware -> auth -> router -> validation
-> endpoint -> service/repository -> database -> serialization -> response
-> browser -> Promise continuation -> JSON parsing -> setState()
-> React reconciliation -> DOM mutation -> style -> layout -> paint -> composite -> pixels
```

Debugging principle

Never debug the entire application at once. Determine the last layer that definitely worked, then inspect the next boundary.

2. Before the click: page delivery and hydration

Before the user can click anything, the page has already travelled from production infrastructure into the browser. For a URL such as <https://app.company.com/users>, a common initial path is:

```
Browser -> DNS -> CDN/reverse proxy -> Next.js server -> HTML/RSC payload + JavaScript -> browser
```

The browser can display HTML before all interactivity is ready. React then connects the rendered interface to client-side JavaScript. In a traditional SSR mental model this is called hydration. In modern Next.js App Router, Server Components and Client Components are combined, but the important question stays the same: which parts must execute in the browser?

Server Components vs Client Components

App Router components are Server Components by default. They can render UI on the server, but browser event handlers such as `onClick` require a Client Component.

```
"use client";

export default function Page() {
  function handleClick() {
    console.log("clicked in the browser");
  }

  return <button onClick={handleClick}>Click</button>;
}
```

Key question

Whenever you see JavaScript in a Next.js project, ask: does this run in the browser, on the Next.js server, on FastAPI, or inside the database?

If a button is visible but clicking does nothing, likely causes include a JavaScript runtime error, failed hydration/client initialization, a missing Client Component boundary, a disabled element, an overlay covering the button, pointer-events CSS, or simply the wrong handler.

3. The physical click, hit testing, and DOM events

A click begins below JavaScript. Mouse or touch hardware reports input to the operating system. The OS routes it to the browser window. The browser renderer then performs hit testing: it asks which rendered element occupies the clicked coordinates.

```
mouse/touch input -> OS -> browser window -> hit testing -> target DOM element
```

The browser creates an event object carrying facts such as event type, target, pointer coordinates, modifier keys, and timestamps. The event then travels through the DOM event propagation model.

Capture -> target -> bubble

```
window
  -> document
    -> body
      -> parent
        -> BUTTON    (capture)

BUTTON                (target)

BUTTON
  -> parent
    -> body
      -> document    (bubble)
```

This explains why a click on a child can trigger a handler on a parent. `stopPropagation()` can prevent propagation when appropriate, but it should be used intentionally because propagation is often useful.

```
<div onClick={() => console.log("parent")}>
  <button onClick={(e) => {
    e.stopPropagation();
    console.log("button");
  }}>
    Delete
  </button>
</div>
```

4. React event handling and the JavaScript event loop

React associates your JSX handler with the rendered element. When the browser event arrives, React invokes the correct handler. At this point nothing has reached FastAPI yet.

```
async function loadUsers() {
  console.log("A: handler started");
  const response = await fetch("/api/users");
  console.log("B: response received", response.status);
}
```

If log A never appears, the problem is before the network: DOM, React, component lifecycle, client execution, CSS, or event propagation. If A appears but B does not, you have moved the problem boundary to the request/network side.

Why await does not freeze the browser

JavaScript starts `fetch()`, then reaches `await`. The async function suspends while the browser networking subsystem continues the request. The JavaScript thread can process other events. When the Promise settles, the continuation is scheduled back onto the event loop.

```
handler runs -> fetch starts network work -> await suspends function -> browser remains responsive -> Promise settles -> continuation runs
```

Important

Async does not mean your JavaScript function runs on another CPU thread. It means the browser can hand off work such as networking and resume your function later.

Part II - The browser-to-server journey

5. fetch(), URLs, and environment variables

fetch() first parses the URL into scheme, host, port, path, and query. For `https://api.example.com/users?page=2`, the browser understands:

```
scheme: https
host:   api.example.com
port:   443
path:   /users
query:  page=2
```

A large class of production bugs comes from building the wrong URL. In Next.js, browser-visible environment variables commonly use the `NEXT_PUBLIC_` prefix. These values can be embedded into the built frontend, so a production build with the wrong value can make every user call the wrong destination.

```
const API_URL = process.env.NEXT_PUBLIC_API_URL;
const response = await fetch(`${API_URL}/users`);
```

localhost trap

localhost always means "this machine" from the point of view of the process using it. In the user browser it means the user machine; in a Docker container it means that container; on the Next.js server it means that server/container.

6. Origins, CORS, preflight, and cookies

What is an origin?

An origin is essentially scheme + host + port. `https://app.company.com` and `https://api.company.com` are different origins because the host differs.

CORS is enforced by browsers

Cross-Origin Resource Sharing controls whether JavaScript from one origin may read responses from another. A backend can return HTTP 200 while browser JavaScript still receives a CORS failure because the browser refuses to expose the response.

```
Browser -> GET /users -> FastAPI -> 200 + Access-Control-Allow-Origin -> Browser decides whether JS may read it
```

Preflight: the hidden OPTIONS request

For requests that are not considered simple, the browser may send OPTIONS before the real request. A POST with Authorization and application/json is a common example.

```
OPTIONS /users
Origin: https://app.company.com
Access-Control-Request-Method: POST
Access-Control-Request-Headers: authorization, content-type
```

If preflight fails, the browser may never send the POST at all. This is why your FastAPI `@app.post("/users")` function can appear to be ignored even though the real failure is CORS configuration.

Cookies and credentials

Browsers attach eligible cookies automatically according to Domain, Path, Secure, SameSite, expiration, and request context. Cross-origin requests may also require credentials: "include" on the frontend and compatible CORS credentials settings on the backend.

```
fetch("https://api.company.com/me", {  
  credentials: "include"  
});
```

A frequent authentication bug is: login succeeds, the cookie exists, the next request omits it, FastAPI receives no session, and the API returns 401.

7. DNS, transport, TLS, and HTTP

DNS

The browser needs an IP address for the hostname. Resolution may use browser cache, OS cache, local resolver, and upstream DNS infrastructure. A DNS failure occurs before your reverse proxy or FastAPI sees any request.

```
api.company.com -> DNS resolution -> IP address
```

TCP/QUIC and TLS

For HTTPS, the browser establishes a transport connection and a secure TLS session. HTTP/1.1 and HTTP/2 normally run over TCP + TLS; HTTP/3 runs over QUIC with TLS integrated. The mental model is: find the server -> establish a connection -> secure it -> exchange HTTP messages.

TLS encrypts the HTTP request in transit. A bearer token may appear as plain text in your application-level header, but it is carried inside the encrypted TLS connection when HTTPS is correctly configured.

HTTP request anatomy

```
POST /users HTTP/1.1  
Host: api.company.com  
Content-Type: application/json  
Authorization: Bearer <token>  
Origin: https://app.company.com  
Cookie: session=<value>  
  
{  
  "name": "Saad"  
}
```

Think of every request as four main ideas: method, path/query, headers, and optional body. When debugging, inspect all four rather than only the URL.

Part III - Production infrastructure and FastAPI

8. Reverse proxies, CDNs, and load balancers

In production, users usually do not connect directly to Uvicorn on port 8000. A public edge layer sits in front. One common architecture is:

```
Internet -> CDN/WAF -> load balancer -> nginx/ingress -> FastAPI instances
```

Another useful architecture presents Next.js and FastAPI under one public origin:

```
https://company.com/      -> Next.js :3000  
https://company.com/api/* -> FastAPI :8000
```

The reverse proxy can terminate TLS, add headers, compress responses, route paths, limit request sizes, and apply timeouts. This means some failures happen before the application.

Proxy-oriented status clues

- 502 Bad Gateway: the proxy could not successfully communicate with the upstream service.

- 503 Service Unavailable: the service is unavailable, overloaded, draining, or has no healthy upstreams.
- 504 Gateway Timeout: the proxy waited too long for an upstream response.

9. Uvicorn, ASGI, middleware, and routing

Uvicorn and ASGI

Uvicorn is an ASGI server. It accepts network connections, parses HTTP, and translates requests into the ASGI protocol understood by FastAPI/Starlette. Your endpoint decorator does not listen on a socket by itself; the ASGI server performs that infrastructure work.

```
socket -> Uvicorn -> ASGI scope/messages -> FastAPI/Starlette
```

Middleware

Middleware wraps the request/response flow. Logging, CORS, authentication, request IDs, timing, tracing, compression, and security policies can all live here.

```
async def middleware(request, call_next):
    # before endpoint
    response = await call_next(request)
    # after endpoint
    return response

request -> middleware A before -> middleware B before -> endpoint -> middleware B after -> middleware A after -> response
```

An endpoint may never execute if middleware returns early. For example, authentication middleware can return 401 before routing to your business code.

Routing

FastAPI matches both path and HTTP method. GET /users and POST /users are different operations. Calling POST against a path that only supports GET commonly returns 405 Method Not Allowed.

Useful distinction

404 usually points toward an unknown path/resource. 405 usually means the path exists but the HTTP method is not allowed.

10. Dependencies, authentication, and validation

Dependency injection

```
@app.get("/users")
async def get_users(
    current_user = Depends(get_current_user),
    db = Depends(get_db),
):
    ...
```

FastAPI resolves dependencies before entering the endpoint. If `get_current_user` raises `HTTPException(401)`, the endpoint body never runs. This gives you another precise boundary to test.

Authentication vs authorization

Authentication asks: who are you? Authorization asks: are you allowed to perform this action? A missing/invalid identity usually maps to 401; an authenticated identity lacking permission commonly maps to 403.

Pydantic validation

```
class UserCreate(BaseModel):
    name: str
    age: int

@app.post("/users")
```

```
async def create_user(user: UserCreate):  
    ...
```

If the browser sends `{"name":"Saad","age":"banana"}`, FastAPI/Pydantic can reject the request before `create_user` executes. In typical FastAPI applications this produces 422 with structured validation detail.

Debugging clue

If you receive 422 and your first print/log inside the endpoint never appears, inspect the request body, query parameters, path parameters, and Pydantic schema.

11. Endpoint, service, repository, and database

A well-structured application often separates transport concerns from business logic and data access:

```
router/controller -> service/use-case -> repository/data access -> database
```

This lets you ask a better question than "is the backend broken?" You can ask whether the failure is in HTTP parsing, authorization, business rules, query construction, or the database itself.

Database journey

The database is usually another service reached over a network connection. For PostgreSQL, a driver/pool such as `asyncpg` or `psycopg` may send SQL. For MongoDB, a driver sends database operations over its protocol.

```
SELECT id, name, email  
FROM users  
WHERE tenant_id = 42  
ORDER BY id  
LIMIT 20;
```

A relational database roughly parses the query, plans it, chooses indexes/access paths, reads data pages, applies filters/joins/sorts, and returns rows. Slow endpoints are often slow because of database behavior rather than React or Python execution.

- Connection refused or DNS failure to the database host.
- Pool exhaustion: every connection is busy, so requests wait.
- Missing index or expensive query plan.
- Locks, deadlocks, or long transactions.
- Constraint failures: unique, foreign key, not-null.
- Query timeout or overloaded database.

Part IV - The trip back to the screen

12. Serialization and the HTTP response

Your Python return value must become bytes suitable for HTTP. FastAPI validates/serializes the value into JSON (or another response type), assigns a status, and sends headers plus the body through ASGI/Uvicorn and the proxy stack.

```
HTTP/1.1 200 OK  
Content-Type: application/json  
Cache-Control: no-store  
  
[{"id":1,"name":"Saad"}]
```

Additional response headers can control CORS, cookies, caching, content encoding, request tracing, and browser security behavior.

13. fetch response behavior and JSON parsing

The browser receives encrypted network bytes, TLS decrypts them, HTTP is parsed, compression may be decoded, cookies/cache/CORS policies are applied, and finally the fetch Promise can settle.

fetch does not reject on every HTTP error

A crucial browser fact: fetch usually resolves when an HTTP response exists, even for 401, 404, 422, or 500. Network-level failures reject the Promise. Therefore check `response.ok` or `response.status` yourself.

```
const response = await fetch("/api/users");

if (!response.ok) {
  throw new Error(`HTTP ${response.status}`);
}

const data = await response.json();
```

The "Unexpected token <" clue

If `response.json()` fails on the character `<`, the server may have returned HTML, often an nginx/hosting error page, a login redirect, or a framework error document. Inspect the Network tab Response directly rather than assuming JSON parsing itself is the root problem.

14. React state, rerenders, and reconciliation

```
const [users, setUsers] = useState<User[]>([]);

setUsers(data);
```

`setUsers` does not directly edit HTML. It schedules a state update. React reruns the relevant component functions to calculate what the UI should look like for the new state. This is a render, not a full page reload.

React then compares the previous and next element trees during reconciliation and commits only the necessary DOM mutations. A component can render many times even when only a small real DOM update is required.

Rerender is normal

Opening a dialog, typing into controlled inputs, changing pagination, or receiving new API data can rerender components. A rerender becomes suspicious when it triggers unintended side effects, repeated network requests, or expensive work.

15. Browser style, layout, paint, and compositing

After React mutates the DOM, the browser still has work to do before the user sees pixels.

```
DOM/CSS changes -> style calculation -> layout -> paint -> compositing -> GPU/display -> pixels
```

Style calculation

The browser computes which CSS rules apply and derives final styles.

Layout

The browser determines geometry: position, width, height, line wrapping, and relationships between boxes.

Paint

Visual primitives such as text, borders, backgrounds, shadows, and images are drawn into paint records/layers.

Compositing

Layers are combined, often with GPU assistance, into the final image displayed on the screen. Performance problems can therefore occur even after JavaScript finishes, for example from layout thrashing or heavy painting.

Part V - Production debugging

16. Common React request bugs

Bug A: effect runs after every render

```
useEffect(() => {  
  fetchUsers();  
}); // no dependency array
```

If `fetchUsers` updates state, the sequence can become render -> effect -> request -> state update -> render -> effect -> request. Depending on the data and component, this can produce repeated or endless traffic.

```
useEffect(() => {  
  fetchUsers();  
}, []); // once after mount (conceptually)
```

Or make dependencies explicit when the request depends on them:

```
useEffect(() => {  
  fetchUsers(page, search);  
}, [page, search]);
```

Bug B: calling the handler during render

```
// Wrong: createUser() runs while rendering  
<Button onClick={createUser()}>Create</Button>  
  
// Correct: pass a function  
<Button onClick={createUser}>Create</Button>  
<Button onClick={() => createUser()}>Create</Button>
```

Bug C: duplicate requests from dev behavior

React development tooling and Strict Mode can intentionally expose unsafe side effects by invoking some lifecycle behavior more than once. Always distinguish development-only diagnostics from true production duplication, and make effects idempotent where possible.

17. Forms and hidden browser behavior

A button inside a form defaults to submit unless you specify another type. This can cause a handler plus a native form submission/navigation, which looks like a mysterious reload.

```
<form onSubmit={handleSubmit}>  
  <button type="submit">Create</button>  
  <button type="button" onClick={closeDialog}>Cancel</button>  
</form>  
  
function handleSubmit(e) {  
  e.preventDefault();  
  // validate and send request  
}
```

When a click unexpectedly navigates, reloads, or sends two actions, always check whether the element is inside a form and whether both `onClick` and `onSubmit` are active.

18. HTTP status-code map

Status	Meaning	First place to think
200	Success	Response is valid; investigate frontend if the UI is wrong.
201	Created	A new resource was created successfully.
204	Success, no body	Do not call <code>response.json()</code> unless a body actually exists.
301/302	Redirect	May alter request flow; inspect

		Location and browser behavior.
307/308	Redirect preserving method	Useful when method/body semantics must be preserved.
400	Bad request	Malformed or semantically unacceptable input.
401	Unauthenticated	Missing/invalid/expired identity or session.
403	Forbidden	Identity known but action is not permitted.
404	Not found	Wrong path, missing resource, or routing mismatch.
405	Method not allowed	Path exists but GET/POST/etc. is wrong.
409	Conflict	State conflict such as duplicate/optimistic concurrency.
422	Validation error	Common FastAPI/Pydantic input mismatch.
429	Too many requests	Rate limiting or quota.
500	Server error	Unhandled backend/application failure.
502	Bad gateway	Proxy/upstream communication failure.
503	Unavailable	No healthy capacity or service temporarily unavailable.
504	Gateway timeout	Proxy timed out waiting for upstream.

19. A systematic debugging algorithm

When a button fails, do not jump randomly between React, FastAPI, and the database. Walk the boundaries in order:

1. Did the click happen? Add a log at the first line of the handler.
2. Did the handler call fetch/axios? Inspect code and console.
3. Did the browser create a network request? Use DevTools Network.
4. Was there an OPTIONS preflight? Did it succeed?
5. Is the URL, method, query, body, and headers exactly what you expected?
6. Did the edge/proxy receive the request?
7. Did FastAPI/Uvicorn receive it?
8. Did middleware or authentication return early?
9. Did routing match the expected endpoint and method?
10. Did Pydantic validation succeed?
11. Did the endpoint body execute?
12. Did the service/repository execute?
13. Did the database receive and finish the query?
14. Did FastAPI return the intended status, headers, and body?
15. Did the browser accept the response or block it because of CORS?
16. Did response parsing succeed?
17. Did state update with the expected data?
18. Did the component rerender?
19. Did React commit the correct DOM changes?
20. Did CSS/layout make the result visible?

Professional debugging habit

Find the first boundary that failed. Do not optimize or rewrite later layers until you can prove the request reached them.

DevTools Network: your primary evidence

Open DevTools -> Network, trigger the action, select the request, then inspect:

- Request URL: is the environment/base URL correct?
- Request Method: GET, POST, PATCH, DELETE, etc.
- Status: which layer does the status code suggest?
- Request Headers: Origin, Authorization, Cookie, Content-Type, Accept.
- Payload: does the body exactly match the backend schema?
- Response: JSON error, HTML proxy page, validation detail, or expected data?
- Timing: DNS, connection, TLS, waiting/TTFB, download.

Console, backend logs, database logs

Console	-> JavaScript/runtime/React/CORS symptoms
Network	-> actual browser HTTP behavior
Proxy logs	-> routing/upstream/TLS/timeout clues
FastAPI logs	-> middleware/validation/endpoint/backend exceptions
Database logs	-> query/locks/connections/performance

20. Performance: where the time actually goes

A slow page is not automatically a React problem. Break total latency into stages and measure them.

```
Example request latency

click handling          1 ms
preflight              30 ms
network to edge        40 ms
proxy/routing          3 ms
authentication         10 ms
database query        1300 ms
serialization          20 ms
network back           40 ms
JSON parsing           5 ms
React render/commit    15 ms
browser paint          10 ms
-----
total                  ~1474 ms
```

In this example, useMemo is irrelevant. The database dominates. Optimize the measured bottleneck: indexes/query plan, caching, payload size, connection reuse, rendering cost, or network topology depending on evidence.

21. Next.js as a middle layer

Not every browser request goes directly to FastAPI. A common production pattern is Browser -> Next.js Route Handler/Server Action -> FastAPI -> database. This can hide private backend addresses, centralize authentication, or simplify same-origin browser behavior.

```
Browser -> Next.js route/server code -> FastAPI -> DB -> FastAPI -> Next.js -> Browser
```

This creates an extra boundary. A request can succeed from Next.js to FastAPI while the browser-to-Next.js step fails, or vice versa. Again ask where each fetch executes.

```
// Executes on the Next.js server when used in a route handler
export async function GET() {
  const r = await fetch("http://backend:8000/users");
  return Response.json(await r.json());
}
```

22. Error-by-layer cheat sheet

Symptom	First layer to inspect
Nothing happens on click	DOM / React / client JavaScript / CSS / disabled element.
Handler logs, no Network request	Handler logic, thrown error before fetch, conditional branch.
OPTIONS fails	CORS/preflight configuration.

ERR_NAME_NOT_RESOLVED	DNS/hostname.
Certificate error	TLS/certificate/domain/expiry.
404	Path/routing/resource.
405	Wrong HTTP method.
401	Authentication/session/token/cookie.
403	Authorization/permissions.
422	FastAPI/Pydantic validation or parameter mismatch.
500	Unhandled backend error.
502	Reverse proxy cannot successfully reach/use upstream.
503	Service unavailable/no healthy capacity.
504	Gateway timeout/upstream too slow.
200 but UI wrong	Parsing/state/rendering/CSS/data-shape assumptions.
API repeats	useEffect dependencies, render-time invocation, retries, query library settings, event duplication.
Page is slow	Use Network Timing + backend tracing + DB query timing; do not guess.

Final mental model

You do not need to memorize every library API to become comfortable in production web development. You need a stable model of the layers and boundaries.

```

USER
-> browser input + DOM
-> React event handler
-> fetch / axios
-> browser security: origin, CORS, cookies
-> DNS + transport + TLS + HTTP
-> CDN / proxy / load balancer
-> Uvicorn + ASGI
-> middleware + auth
-> router + validation
-> endpoint + business logic
-> database
-> serialization + HTTP response
-> browser fetch + JSON
-> React state + render + reconciliation
-> DOM + style + layout + paint + composite
-> pixels

```

The question to keep asking

What machine am I on? What layer am I in? What data is crossing this boundary? What should happen next? What evidence proves it happened?

Once you think this way, repeated Axios calls, CORS failures, wrong .env URLs, 422 validation errors, 502 proxy failures, authentication bugs, slow pages, and unexpected rerenders stop looking like unrelated magic. They become failures at identifiable points in one continuous pipeline.

Appendix - Minimal instrumentation template

Frontend:

```

async function loadUsers() {
  console.log("[1] click");

  try {
    console.log("[2] sending request");
    const res = await fetch("/api/users");
    console.log("[3] response", res.status);

    const text = await res.text();
    console.log("[4] raw body", text);

    if (!res.ok) throw new Error(`HTTP ${res.status}`);
  }
}

```

```
const data = JSON.parse(text);
console.log("[5] parsed data", data);
setUsers(data);
console.log("[6] state update scheduled");
} catch (err) {
  console.error("[ERROR]", err);
}
}
```

Backend:

```
@app.get("/users")
async def users():
  logger.info("[7] endpoint entered")
  users = await repo.get_users()
  logger.info("[8] database returned", extra={"count": len(users)})
  return users
```

This is not a replacement for proper structured logging/tracing, but it teaches the most important debugging skill: determine the last confirmed point in the journey.